

You can also get an instance of the [SystemMediaTransportControls](#) by calling [GetForCurrentView](#). You must get the object with this method if you are using [MediaElement](#) to play media.

C#

```
_systemMediaTransportControls =  
SystemMediaTransportControls.GetForCurrentView();
```

Enable the buttons that your app will use by setting the corresponding "is enabled" property of the [SystemMediaTransportControls](#) object, such as [IsPlayEnabled](#), [IsPauseEnabled](#), [IsNextEnabled](#), and [IsPreviousEnabled](#). See the [SystemMediaTransportControls](#) reference documentation for a complete list of available controls.

C#

```
_systemMediaTransportControls.IsPlayEnabled = true;  
_systemMediaTransportControls.IsPauseEnabled = true;
```

Register a handler for the [ButtonPressed](#) event to receive notifications when the user presses a button.

C#

```
_systemMediaTransportControls.ButtonPressed += SystemControls_ButtonPressed;
```

Handle system media transport controls button presses

The [ButtonPressed](#) event is raised by the system transport controls when one of the enabled buttons is pressed. The [Button](#) property of the [SystemMediaTransportControlsButtonPressedEventArgs](#) passed into the event handler is a member of the [SystemMediaTransportControlsButton](#) enumeration that indicates which of the enabled buttons was pressed.

In order to update objects on the UI thread from the [ButtonPressed](#) event handler, such as a [MediaElement](#) object, you must marshal the calls through the [CoreDispatcher](#). This is because the [ButtonPressed](#) event handler is not called from the UI thread and therefore an exception will be thrown if you attempt to modify the UI directly.

C#

```

async void SystemControls_ButtonPressed(SystemMediaTransportControls sender,
    SystemMediaTransportControlsButtonPressedEventArgs args)
{
    switch (args.Button)
    {
        case SystemMediaTransportControlsButton.Play:
            await
Dispatcher.RunAsync(Windows.UI.Core.CoreDispatcherPriority.Normal, () =>
            {
                mediaElement.Play();
            });
            break;
        case SystemMediaTransportControlsButton.Pause:
            await
Dispatcher.RunAsync(Windows.UI.Core.CoreDispatcherPriority.Normal, () =>
            {
                mediaElement.Pause();
            });
            break;
        default:
            break;
    }
}

```

Update the system media transport controls with the current media status

You should notify the [SystemMediaTransportControls](#) when the state of the media has changed so that the system can update the controls to reflect the current state. To do this, set the [PlaybackStatus](#) property to the appropriate [MediaPlaybackStatus](#) value from within the [CurrentStateChanged](#) event of the [MediaElement](#), which is raised when the media state changes.

C#

```

void MediaElement_CurrentStateChanged(object sender, RoutedEventArgs e)
{
    switch (mediaElement.CurrentState)
    {
        case MediaElementState.Playing:
            _systemMediaTransportControls.PlaybackStatus =
MediaPlaybackStatus.Playing;
            break;
        case MediaElementState.Paused:
            _systemMediaTransportControls.PlaybackStatus =
MediaPlaybackStatus.Paused;
            break;
        case MediaElementState.Stopped:

```

```

        _systemMediaTransportControls.PlaybackStatus =
MediaPlaybackStatus.Stopped;
        break;
    case MediaElementState.Closed:
        _systemMediaTransportControls.PlaybackStatus =
MediaPlaybackStatus.Closed;
        break;
    default:
        break;
    }
}

```

Update the system media transport controls with media info and thumbnails

Use the [SystemMediaTransportControlsDisplayUpdater](#) class to update the media info that is displayed by the transport controls, such as the song title or the album art for the currently playing media item. Get an instance of this class with the [SystemMediaTransportControls.DisplayUpdater](#) property. For typical scenarios, the recommended way to pass the metadata is to call [CopyFromFileAsync](#), passing in the currently playing media file. The display updater will automatically extract the metadata and thumbnail image from the file.

Call the [Update](#) to cause the system media transport controls to update its UI with the new metadata and thumbnail.

C#

```

async void MediaElement_MediaOpened(object sender, RoutedEventArgs e)
{
    // Get the updater.
    SystemMediaTransportControlsDisplayUpdater updater =
_systemMediaTransportControls.DisplayUpdater;

    await updater.CopyFromFileAsync(MediaPlaybackType.Music,
currentMediaFile);

    // Update the system media transport controls
    updater.Update();
}

```

If your scenario requires it, you can update the metadata displayed by the system media transport controls manually by setting the values of the [MusicProperties](#), [ImageProperties](#), or [VideoProperties](#) objects exposed by the [DisplayUpdater](#) class.

C#

```
// Get the updater.
SystemMediaTransportControlsDisplayUpdater updater =
_systemMediaTransportControls.DisplayUpdater;

// Music metadata.
updater.Type = MediaPlayerType.Music;
updater.MusicProperties.Artist = "artist";
updater.MusicProperties.AlbumArtist = "album artist";
updater.MusicProperties.Title = "song title";

// Set the album art thumbnail.
// RandomAccessStreamReference is defined in Windows.Storage.Streams
updater.Thumbnail =
    RandomAccessStreamReference.CreateFromUri(new Uri("ms-
appx:///Music/music1_AlbumArt.jpg"));

// Update the system media transport controls.
updater.Update();
```

⚠ Note

Apps should set a value for the [SystemMediaTransportControlsDisplayUpdater.Type](#) property even if they aren't supplying other media metadata to be displayed by the System Media Transport Controls. This value helps the system handle your media content correctly, including preventing the screen saver from activating during playback.

Update the system media transport controls timeline properties

The system transport controls display information about the timeline of the currently playing media item, including the current playback position, the start time, and the end time of the media item. To update the system transport controls timeline properties, create a new [SystemMediaTransportControlsTimelineProperties](#) object. Set the properties of the object to reflect the current state of the playing media item. Call [SystemMediaTransportControls.UpdateTimelineProperties](#) to cause the controls to update the timeline.

C#

```
// Create our timeline properties object
var timelineProperties = new
SystemMediaTransportControlsTimelineProperties();
```

```
// Fill in the data, using the media elements properties
timelineProperties.StartTime = TimeSpan.FromSeconds(0);
timelineProperties.MinSeekTime = TimeSpan.FromSeconds(0);
timelineProperties.Position = mediaElement.Position;
timelineProperties.MaxSeekTime = mediaElement.NaturalDuration.TimeSpan;
timelineProperties.EndTime = mediaElement.NaturalDuration.TimeSpan;

// Update the System Media transport Controls
_systemMediaTransportControls.UpdateTimelineProperties(timelineProperties);
```

- You must provide a value for the **StartTime**, **EndTime** and **Position** in order for the system controls to display a timeline for your playing item.
- **MinSeekTime** and **MaxSeekTime** allow you to specify the range within the timeline that the user can seek. A typical scenario for this is to allow content providers to include advertisement breaks in their media.

You must set **MinSeekTime** and **MaxSeekTime** in order for the **PositionChangeRequest** to be raised.

- It is recommended that you keep the system controls in sync with your media playback by updating these properties approximately every 5 seconds during playback and again whenever the state of playback changes, such as pausing or seeking to a new position.

Respond to player property changes

There is a set of system transport controls properties that relate to the current state of the media player itself, rather than the state of the playing media item. Each of these properties is matched with an event that is raised when the user adjusts the associated control. These properties and events include:

 Expand table

Property	Event
AutoRepeatMode	AutoRepeatModeChangeRequested
PlaybackRate	PlaybackRateChangeRequested
ShuffleEnabled	ShuffleEnabledChangeRequested

To handle user interaction with one of these controls, first register a handler for the associated event.

C#

```
_systemMediaTransportControls.PlaybackRateChangeRequested +=  
SystemControls_PlaybackRateChangeRequested;
```

In the handler for the event, first make sure that the requested value is within a valid and expected range. If it is, set the corresponding property on [MediaElement](#) and then set the corresponding property on the [SystemMediaTransportControls](#) object.

C#

```
void SystemControls_PlaybackRateChangeRequested(SystemMediaTransportControls  
sender, PlaybackRateChangeRequestedEventArgs args)  
{  
    // Check the requested value to make sure it is within a valid and  
    // expected range  
    if (args.RequestedPlaybackRate >= 0 && args.RequestedPlaybackRate <= 2)  
    {  
        // Set the requested value on the MediaElement  
        mediaElement.PlaybackRate = args.RequestedPlaybackRate;  
  
        // Update the system media controls to reflect the new value  
        _systemMediaTransportControls.PlaybackRate =  
mediaElement.PlaybackRate;  
    }  
}
```

- In order for one of these player property events to be raised, you must set an initial value for the property. For example, [PlaybackRateChangeRequested](#) will not be raised until after you have set a value for the [PlaybackRate](#) property at least one time.

Use the system media transport controls for background audio

If you are not using the automatic SMTC integration provided by [MediaPlayer](#) you must manually integrate with the SMTC to enable background audio. At a minimum, your app must enable the play and pause buttons by setting [IsPlayEnabled](#) and [IsPauseEnabled](#) to true. Your app must also handle the [ButtonPressed](#) event. If your app does not meet these requirements, audio playback will stop when your app moves to the background.

Apps that use the new one-process model for background audio should get an instance of the [SystemMediaTransportControls](#) by calling [GetForCurrentView](#). Apps that use the legacy two-process model for background audio must use

`BackgroundMediaPlayer.Current.SystemMediaTransportControls` to get access to the SMTC from their background process.

For more information on playing audio in the background, see [Play media in the background](#).

Related topics

- [Media playback](#)
- [Integrate with the System Media Transport Controls](#)
- [System Media Transport sample](#) 

Feedback

Was this page helpful?

 Yes

 No

[Provide product feedback](#)  | [Get help at Microsoft Q&A](#)

System-supported timed metadata cues

Article • 03/21/2023

This article describes how to take advantage of several formats of timed metadata that may be embedded in media files or streams. UWP apps can register for events that are raised by the media pipeline during playback whenever these metadata cues are encountered. Using the [DataCue](#) class, apps can implement their own custom metadata cues, but this article focuses on several metadata standards that are automatically detected by the media pipeline, including:

- Image-based subtitles in VobSub format
- Speech cues, including word boundaries, sentence boundaries, and Speech Synthesis Markup Language (SSML) bookmarks
- Chapter cues
- Extended M3U comments
- ID3 tags
- Fragmented mp4 emsg boxes

This article builds on the concepts discussed in the article [Media items, playlists, and tracks](#), which includes the basics of working with the [MediaSource](#), [MediaPlaybackItem](#), and [TimedMetadataTrack](#) classes and general guidance for using timed metadata in your app.

The basic implementation steps are the same for all of the different types of timed metadata described in this article:

1. Create a [MediaSource](#) and then a [MediaPlaybackItem](#) for the content to be played.
2. Register for the [MediaPlaybackItem.TimedMetadataTracksChanged](#) event, which occurs as the sub-tracks of the media item are resolved by the media pipeline.
3. Register for the [TimedMetadataTrack.CueEntered](#) and [TimedMetadataTrack.CueExited](#) events for the timed metadata tracks you want to use.
4. In the **CueEntered** event handler, update your UI based on the metadata passed in the event args. You can update the UI again, to remove the current subtitle text for example, in the **CueExited** event.

In this article, handling each type of metadata is shown as a distinct scenario, but it's possible to handle (or ignore) different types of metadata using mostly shared code. You can check the [TimedMetadataKind](#) property of the [TimedMetadataTrack](#) object at multiple points in the process. So, for example, you could choose to register for the

CueEntered event for metadata tracks that have the value **TimedMetadataKind.ImageSubtitle**, but not for tracks that have the value **TimedMetadataKind.Speech**. Or instead, you could register a handler for all metadata track types and then check the **TimedMetadataKind** value inside the **CueEntered** handler to determine what action to take in response to the cue.

Image-based subtitles

Starting with Windows 10, version 1703, UWP apps can support external image-based subtitles in VobSub format. To use this feature, first create a **MediaSource** object for the media content for which image subtitles will be displayed. Next, create a **TimedTextSource** object by calling **CreateFromUriWithIndex** or **CreateFromStreamWithIndex**, passing in the Uri of the .sub file containing the subtitle image data and the .idx file containing the timing information for the subtitles. Add the **TimedTextSource** to the **MediaSource** by adding it to the source's **ExternalTimedTextSources** collection. Create a **MediaPlaybackItem** from the **MediaSource**.

C#

```
var contentUri = new Uri("http://contoso.com/content.mp4");
var mediaSource = MediaSource.CreateFromUri(contentUri);

var subUri = new Uri("http://contoso.com/content.sub");
var idxUri = new Uri("http://contoso.com/content.idx");
var timedTextSource = TimedTextSource.CreateFromUriWithIndex(subUri,
    idxUri);
mediaSource.ExternalTimedTextSources.Add(timedTextSource);

var mediaPlaybackItem = new MediaPlaybackItem(mediaSource);
```

Register for the image subtitle metadata events using the **MediaPlaybackItem** object created in the previous step. This example uses a helper method, **RegisterMetadataHandlerForImageSubtitles**, to register for the events. A lambda expression is used to implement a handler for the **TimedMetadataTracksChanged** event, which occurs when the system detects a change in the metadata tracks associated with a **MediaPlaybackItem**. In some cases, the metadata tracks may be available when the playback item is initially resolved, so outside of the **TimedMetadataTracksChanged** handler, we also loop through the available metadata tracks and call **RegisterMetadataHandlerForImageSubtitles**.

C#

```

mediaPlaybackItem.TimedMetadataTracksChanged += (MediaPlaybackItem sender,
IVectorChangedEventArgs args) =>
{
    if (args.CollectionChange == CollectionChange.ItemInserted)
    {
        RegisterMetadataHandlerForImageSubtitles(sender, (int)args.Index);
    }
    else if (args.CollectionChange == CollectionChange.Reset)
    {
        for (int index = 0; index < sender.TimedMetadataTracks.Count;
index++)
        {
            if (sender.TimedMetadataTracks[index].TimedMetadataKind ==
TimedMetadataKind.ImageSubtitle)
                RegisterMetadataHandlerForImageSubtitles(sender, index);
        }
    }
};

for (int index = 0; index < mediaPlaybackItem.TimedMetadataTracks.Count;
index++)
{
    RegisterMetadataHandlerForImageSubtitles(mediaPlaybackItem, index);
}

```

After registering for the image subtitle metadata events, the **MediaItem** is assigned to a **MediaPlayer** for playback within a **MediaPlayerElement**.

C#

```

_mediaPlayer = new MediaPlayer();
mediaPlayerElement.SetMediaPlayer(_mediaPlayer);
_mediaPlayer.Source = mediaPlaybackItem;
_mediaPlayer.Play();

```

In the **RegisterMetadataHandlerForImageSubtitles** helper method, get an instance of the **TimedMetadataTrack** class by indexing into the **TimedMetadataTracks** collection of the **MediaPlaybackItem**. Register for the **CueEntered** event and the **CueExited** event. Then, you must call **SetPresentationMode** on the playback item's **TimedMetadataTracks** collection, to instruct the system that the app wants to receive metadata cue events for this playback item.

C#

```

private void RegisterMetadataHandlerForImageSubtitles(MediaPlaybackItem
item, int index)
{
    var timedTrack = item.TimedMetadataTracks[index];
    timedTrack.CueEntered += metadata_ImageSubtitleCueEntered;
}

```

```

        timedTrack.CueExited += metadata_ImageSubtitleCueExited;
        item.TimedMetadataTracks.SetPresentationMode((uint)index,
TimedMetadataTrackPresentationMode.ApplicationPresented);

    }

```

In the handler for the **CueEntered** event, you can check the **TimedMetadataKind** property of the **TimedMetadataTrack** object passed into the handler to see if the metadata is for image subtitles. This is necessary if you are using the same data cue event handler for multiple types of metadata. If the associated metadata track is of type **TimedMetadataKind.ImageSubtitle**, cast the data cue contained in the **Cue** property of the **MediaCueEventArgs** to an **ImageCue**. The **SoftwareBitmap** property of the **ImageCue** contains a **SoftwareBitmap** representation of the subtitle image. Create a **SoftwareBitmapSource** and call **SetBitmapAsync** to assign the image to a XAML **Image** control. The **Extent** and **Position** properties of the **ImageCue** provide information about the size and position of the subtitle image.

C#

```

private async void metadata_ImageSubtitleCueEntered(TimedMetadataTrack
timedMetadataTrack, MediaCueEventArgs args)
{
    // Check in case there are different tracks and the handler was used for
    more tracks
    if (timedMetadataTrack.TimedMetadataKind ==
TimedMetadataKind.ImageSubtitle)
    {
        var cue = args.Cue as ImageCue;
        if (cue != null)
        {
            await
Dispatcher.RunAsync(Windows.UI.Core.CoreDispatcherPriority.Normal, async ()
=>
            {
                var source = new SoftwareBitmapSource();
                await source.SetBitmapAsync(cue.SoftwareBitmap);
                SubtitleImage.Source = source;
                SubtitleImage.Width = cue.Extent.Width;
                SubtitleImage.Height = cue.Extent.Height;
                SubtitleImage.SetValue(Canvas.LeftProperty, cue.Position.X);
                SubtitleImage.SetValue(Canvas.TopProperty, cue.Position.Y);
            });
        }
    }
}

```

Speech cues

Starting with Windows 10, version 1703, UWP apps can register to receive events in response to word boundaries, sentence boundaries, and Speech Synthesis Markup Language (SSML) bookmarks in played media. This allows you to play audio streams generated with the [SpeechSynthesizer](#) class and update your UI based on these events, such as displaying the text of the currently playing word or sentence.

The example shown in this section uses a class member variable to store a text string that will be synthesized and played back.

C#

```
string inputText = "In the lake heading for the mountain, the flea swims";
```

Create a new instance of the [SpeechSynthesizer](#) class. Set the [IncludeWordBoundaryMetadata](#) and [IncludeSentenceBoundaryMetadata](#) options for the synthesizer to **true** to specify that the metadata should be included in the generated media stream. Call [SynthesizeTextToStreamAsync](#) to generate a stream containing the synthesized speech and corresponding metadata. Create a [MediaSource](#) and a [MediaPlaybackItem](#) from the synthesized stream.

C#

```
var synthesizer = new Windows.Media.SpeechSynthesis.SpeechSynthesizer();

// Enable word marker generation (false by default).
synthesizer.Options.IncludeWordBoundaryMetadata = true;
synthesizer.Options.IncludeSentenceBoundaryMetadata = true;

var stream = await synthesizer.SynthesizeTextToStreamAsync(inputText);
var mediaSource = MediaSource.CreateFromStream(stream, "");
var mediaPlaybackItem = new MediaPlaybackItem(mediaSource);
```

Register for the speech metadata events using the [MediaPlaybackItem](#) object. This example uses a helper method, [RegisterMetadataHandlerForSpeech](#), to register for the events. A lambda expression is used to implement a handler for the [TimedMetadataTracksChanged](#) event, which occurs when the system detects a change in the metadata tracks associated with a [MediaPlaybackItem](#). In some cases, the metadata tracks may be available when the playback item is initially resolved, so outside of the [TimedMetadataTracksChanged](#) handler, we also loop through the available metadata tracks and call [RegisterMetadataHandlerForSpeech](#).

C#

```
// Since the tracks are added later we will
// monitor the tracks being added and subscribe to the ones of interest
```

```

mediaPlaybackItem.TimedMetadataTracksChanged += (MediaPlayerItem sender,
IVectorChangedEventArgs args) =>
{
    if (args.CollectionChange == CollectionChange.ItemInserted)
    {
        RegisterMetadataHandlerForSpeech(sender, (int)args.Index);
    }
    else if (args.CollectionChange == CollectionChange.Reset)
    {
        for (int index = 0; index < sender.TimedMetadataTracks.Count;
index++)
        {
            RegisterMetadataHandlerForSpeech(sender, index);
        }
    }
};

// If tracks were available at source resolution time, iterate through and
register:
for (int index = 0; index < mediaPlaybackItem.TimedMetadataTracks.Count;
index++)
{
    RegisterMetadataHandlerForSpeech(mediaPlaybackItem, index);
}

```

After registering for the speech metadata events, the **MediaItem** is assigned to a **MediaPlayer** for playback within a **MediaPlayerElement**.

C#

```

_mediaPlayer = new MediaPlayer();
mediaPlayerElement.SetMediaPlayer(_mediaPlayer);
_mediaPlayer.Source = mediaPlaybackItem;
_mediaPlayer.Play();

```

In the **RegisterMetadataHandlerForSpeech** helper method, get an instance of the **TimedMetadataTrack** class by indexing into the **TimedMetadataTracks** collection of the **MediaPlaybackItem**. Register for the **CueEntered** event and the **CueExited** event. Then, you must call **SetPresentationMode** on the playback item's **TimedMetadataTracks** collection, to instruct the system that the app wants to receive metadata cue events for this playback item.

C#

```

private void RegisterMetadataHandlerForSpeech(MediaPlaybackItem item, int
index)
{
    var timedTrack = item.TimedMetadataTracks[index];
    timedTrack.CueEntered += metadata_SpeechCueEntered;
    timedTrack.CueExited += metadata_SpeechCueExited;
}

```

```

        item.TimedMetadataTracks.SetPresentationMode((uint)index,
        TimedMetadataTrackPresentationMode.ApplicationPresented);

    }

```

In the handler for the **CueEntered** event, you can check the **TimedMetadataKind** property of the **TimedMetadataTrack** object passed into the handler to see if the metadata is speech. This is necessary if you are using the same data cue event handler for multiple types of metadata. If the associated metadata track is of type **TimedMetadataKind.Speech**, cast the data cue contained in the **Cue** property of the **MediaCueEventArgs** to a **SpeechCue**. For speech cues, the type of speech cue included in the metadata track is determined by checking the **Label** property. The value of this property will be "SpeechWord" for word boundaries, "SpeechSentence" for sentence boundaries, or "SpeechBookmark" for SSML bookmarks. In this example, we check for the "SpeechWord" value, and if this value is found, the **StartPositionInInput** and **EndPositionInInput** properties of the **SpeechCue** are used to determine location within the input text of the word currently being played back. This example simply outputs each word to the debug output.

C#

```

private void metadata_SpeechCueEntered(TimedMetadataTrack
timedMetadataTrack, MediaCueEventArgs args)
{
    // Check in case there are different tracks and the handler was used for
    // more tracks
    if (timedMetadataTrack.TimedMetadataKind == TimedMetadataKind.Speech)
    {
        var cue = args.Cue as SpeechCue;
        if (cue != null)
        {
            if (timedMetadataTrack.Label == "SpeechWord")
            {
                // Do something with the cue
                System.Diagnostics.Debug.WriteLine($"
{cue.StartPositionInInput} - {cue.EndPositionInInput}:
{inputText.Substring((int)cue.StartPositionInInput,
((int)cue.EndPositionInInput - (int)cue.StartPositionInInput) + 1)}");
            }
        }
    }
}

```

Chapter cues

Starting with Windows 10, version 1703, UWP apps can register for cues that correspond to chapters within a media item. To use this feature, create a **MediaSource** object for the media content and then create a **MediaPlaybackItem** from the **MediaSource**.

C#

```
var contentUri = new Uri("http://contoso.com/content.mp4");
var mediaSource = MediaSource.CreateFromUri(contentUri);
var mediaPlaybackItem = new MediaPlaybackItem(mediaSource);
```

Register for the chapter metadata events using the **MediaPlaybackItem** object created in the previous step. This example uses a helper method, **RegisterMetadataHandlerForChapterCues**, to register for the events. A lambda expression is used to implement a handler for the **TimedMetadataTracksChanged** event, which occurs when the system detects a change in the metadata tracks associated with a **MediaPlaybackItem**. In some cases, the metadata tracks may be available when the playback item is initially resolved, so outside of the **TimedMetadataTracksChanged** handler, we also loop through the available metadata tracks and call **RegisterMetadataHandlerForChapterCues**.

C#

```
mediaPlaybackItem.TimedMetadataTracksChanged += (MediaPlaybackItem sender,
IVectorChangedEventArgs args) =>
{
    if (args.CollectionChange == CollectionChange.ItemInserted)
    {
        RegisterMetadataHandlerForChapterCues(sender, (int)args.Index);
    }
    else if (args.CollectionChange == CollectionChange.Reset)
    {
        for (int index = 0; index < sender.TimedMetadataTracks.Count;
index++)
        {
            if (sender.TimedMetadataTracks[index].TimedMetadataKind ==
TimedMetadataKind.ImageSubtitle)
                RegisterMetadataHandlerForChapterCues(sender, index);
        }
    }
};

for (int index = 0; index < mediaPlaybackItem.TimedMetadataTracks.Count;
index++)
{
    RegisterMetadataHandlerForChapterCues(mediaPlaybackItem, index);
}
```

After registering for the chapter metadata events, the **MediaItem** is assigned to a **MediaPlayer** for playback within a **MediaPlayerElement**.

C#

```
_mediaPlayer = new MediaPlayer();
mediaPlayerElement.SetMediaPlayer(_mediaPlayer);
_mediaPlayer.Source = mediaPlaybackItem;
_mediaPlayer.Play();
```

In the **RegisterMetadataHandlerForChapterCues** helper method, get an instance of the **TimedMetadataTrack** class by indexing into the **TimedMetadataTracks** collection of the **MediaPlaybackItem**. Register for the **CueEntered** event and the **CueExited** event. Then, you must call **SetPresentationMode** on the playback item's **TimedMetadataTracks** collection, to instruct the system that the app wants to receive metadata cue events for this playback item.

C#

```
private void RegisterMetadataHandlerForChapterCues(MediaPlaybackItem item,
int index)
{
    var timedTrack = item.TimedMetadataTracks[index];
    timedTrack.CueEntered += metadata_ChapterCueEntered;
    timedTrack.CueExited += metadata_ChapterCueExited;
    item.TimedMetadataTracks.SetPresentationMode((uint)index,
TimedMetadataTrackPresentationMode.ApplicationPresented);
}
```

In the handler for the **CueEntered** event, you can check the **TimedMetadataKind** property of the **TimedMetadataTrack** object passed into the handler to see if the metadata is for chapter cues. This is necessary if you are using the same data cue event handler for multiple types of metadata. If the associated metadata track is of type **TimedMetadataKind.Chapter**, cast the data cue contained in the **Cue** property of the **MediaCueEventArgs** to an **ChapterCue**. The **Title** property of the **ChapterCue** contains the title of the chapter that has just been reached in playback.

C#

```
private async void metadata_ChapterCueEntered(TimedMetadataTrack
timedMetadataTrack, MediaCueEventArgs args)
{
    // Check in case there are different tracks and the handler was used for
    more tracks
    if (timedMetadataTrack.TimedMetadataKind == TimedMetadataKind.Chapter)
    {
        var cue = args.Cue as ChapterCue;
```



```

        if (cue != null)
        {
            await
            Dispatcher.RunAsync(Windows.UI.Core.CoreDispatcherPriority.Normal, () =>
            {
                ChapterTitleTextBlock.Text = cue.Title;
            });
        }
    }
}

```

Seek to the next chapter using chapter cues

In addition to receiving notifications when the current chapter changes in a playing item, you can also use chapter cues to seek to the next chapter within a playing item. The example method shown below takes as arguments a [MediaPlayer](#) and a [MediaPlaybackItem](#) representing the currently playing media item. The [TimedMetadataTracks](#) collection is searched to see if any of the tracks have [TimedMetadataKind](#) property of the [TimedMetadataTrack](#) value of [TimedMetadataKind.Chapter](#). If a chapter track is found, the method loops through each cue in the track's [Cues](#) collection to find the first cue that has a [StartTime](#) greater than the current [Position](#) of the media player's playback session. Once the correct cue is found, the position of the playback session is updated and the chapter title is updated in the UI.

C#

```

private void GoToNextChapter(MediaPlayer player, MediaPlaybackItem item)
{
    // Find the chapters track if one exists
    TimedMetadataTrack chapterTrack =
    item.TimedMetadataTracks.FirstOrDefault(track => track.TimedMetadataKind ==
    TimedMetadataKind.Chapter);
    if (chapterTrack == null)
    {
        return;
    }

    // Find the first chapter that starts after current playback position
    TimeSpan currentPosition = player.PlaybackSession.Position;
    foreach (ChapterCue cue in chapterTrack.Cues)
    {
        if (cue.StartTime > currentPosition)
        {
            // Change player position to chapter start time
            player.PlaybackSession.Position = cue.StartTime;

            // Display chapter name

```

```

        ChapterTitleTextBlock.Text = cue.Title;
        break;
    }
}
}

```

Extended M3U comments

Starting with Windows 10, version 1703, UWP apps can register for cues that correspond to comments within a Extended M3U manifest file. This example uses [AdaptiveMediaSource](#) to play the media content. For more information, see [Adaptive Streaming](#). Create an [AdaptiveMediaSource](#) for the content by calling [CreateFromUriAsync](#) or [CreateFromStreamAsync](#). Create a [MediaSource](#) object by calling [CreateFromAdaptiveMediaSource](#) and then create a [MediaPlaybackItem](#) from the [MediaSource](#).

C#

```

AdaptiveMediaSourceCreationResult result =
    await AdaptiveMediaSource.CreateFromUriAsync(new
Uri("http://contoso.com/playlist.m3u"));

if (result.Status != AdaptiveMediaSourceCreationStatus.Success)
{
    // TODO: Handle adaptive media source creation errors.
    return;
}
var mediaSource =
MediaSource.CreateFromAdaptiveMediaSource(result.MediaSource);
var mediaPlaybackItem = new MediaPlaybackItem(mediaSource);

```

Register for the M3U metadata events using the [MediaPlaybackItem](#) object created in the previous step. This example uses a helper method, [RegisterMetadataHandlerForEXTM3UCues](#), to register for the events. A lambda expression is used to implement a handler for the [TimedMetadataTracksChanged](#) event, which occurs when the system detects a change in the metadata tracks associated with a [MediaPlaybackItem](#). In some cases, the metadata tracks may be available when the playback item is initially resolved, so outside of the [TimedMetadataTracksChanged](#) handler, we also loop through the available metadata tracks and call [RegisterMetadataHandlerForEXTM3UCues](#).

C#

```

mediaPlaybackItem.TimedMetadataTracksChanged += (MediaPlaybackItem sender,
IVectorChangedEventArgs args) =>

```

```

{
    if (args.CollectionChange == CollectionChange.ItemInserted)
    {
        RegisterMetadataHandlerForEXTM3UCues(sender, (int)args.Index);
    }
    else if (args.CollectionChange == CollectionChange.Reset)
    {
        for (int index = 0; index < sender.TimedMetadataTracks.Count;
index++)
        {
            if (sender.TimedMetadataTracks[index].TimedMetadataKind ==
TimedMetadataKind.ImageSubtitle)
                RegisterMetadataHandlerForEXTM3UCues(sender, index);
        }
    }
};

for (int index = 0; index < mediaPlayerItem.TimedMetadataTracks.Count;
index++)
{
    RegisterMetadataHandlerForEXTM3UCues(mediaPlaybackItem, index);
}

```

After registering for the M3U metadata events, the **MediaItem** is assigned to a **MediaPlayer** for playback within a **MediaPlayerElement**.

C#

```

_mediaPlayer = new MediaPlayer();
mediaPlayerElement.SetMediaPlayer(_mediaPlayer);
_mediaPlayer.Source = mediaPlaybackItem;
_mediaPlayer.Play();

```

In the **RegisterMetadataHandlerForEXTM3UCues** helper method, get an instance of the **TimedMetadataTrack** class by indexing into the **TimedMetadataTracks** collection of the **MediaPlaybackItem**. Check the **DispatchType** property of the metadata track, which will have a value of "EXTM3U" if the track represents M3U comments. Register for the **CueEntered** event and the **CueExited** event. Then, you must call **SetPresentationMode** on the playback item's **TimedMetadataTracks** collection, to instruct the system that the app wants to receive metadata cue events for this playback item.

C#

```

private void RegisterMetadataHandlerForEXTM3UCues(MediaPlaybackItem item,
int index)
{
    var timedTrack = item.TimedMetadataTracks[index];
    var dispatchType = timedTrack.DispatchType;

```

```

        if (String.Equals(dispatchType, "EXTM3U",
StringComparison.OrdinalIgnoreCase))
        {
            timedTrack.Label = "EXTM3U comments";
            timedTrack.CueEntered += metadata_EXTM3UCueEntered;
            timedTrack.CueExited += metadata_EXTM3UCueExited;
            item.TimedMetadataTracks.SetPresentationMode((uint)index,
TimedMetadataTrackPresentationMode.ApplicationPresented);
        }
    }
}

```

In the handler for the **CueEntered** event, cast the data cue contained in the **Cue** property of the **MediaCueEventArgs** to an **DataCue**. Check to make sure the **DataCue** and the **Data** property of the cue are not null. Extended EMU comments are provided in the form of UTF-16, little endian, null terminated strings. Create a new **DataReader** to read the cue data by calling **DataReader.FromBuffer**. Set the **UnicodeEncoding** property of the reader to **Utf16LE** to read the data in the correct format. Call **ReadString** to read the data, specifying half of the length of the **Data** field, because each character is two bytes in size, and subtract one to remove the trailing null character. In this example, the M3U comment is simply written to the debug output.

C#

```

private void metadata_EXTM3UCueEntered(TimedMetadataTrack
timedMetadataTrack, MediaCueEventArgs args)
{
    var dataCue = args.Cue as DataCue;
    if (dataCue != null && dataCue.Data != null)
    {
        // The payload is a UTF-16 Little Endian null-terminated string.
        // It is any comment line in a manifest that is not part of the HLS
spec.
        var dr =
Windows.Storage.Streams.DataReader.FromBuffer(dataCue.Data);
        dr.UnicodeEncoding =
Windows.Storage.Streams.UnicodeEncoding.Utf16LE;
        var m3uComment = dr.ReadString(dataCue.Data.Length / 2 - 1);
        System.Diagnostics.Debug.WriteLine(m3uComment);
    }
}

```

ID3 tags

Starting with Windows 10, version 1703, UWP apps can register for cues that correspond to ID3 tags within Http Live Streaming (HLS) content. This example uses **AdaptiveMediaSource** to play the media content. For more information, see **Adaptive Streaming**. Create an **AdaptiveMediaSource** for the content by calling

[CreateFromUriAsync](#) or [CreateFromStreamAsync](#). Create a [MediaSource](#) object by calling [CreateFromAdaptiveMediaSource](#) and then create a [MediaPlaybackItem](#) from the [MediaSource](#).

C#

```
AdaptiveMediaSourceCreationResult result =  
    await AdaptiveMediaSource.CreateFromUriAsync(new  
Uri("http://contoso.com/playlist.m3u"));  
  
if (result.Status != AdaptiveMediaSourceCreationStatus.Success)  
{  
    // TODO: Handle adaptive media source creation errors.  
    return;  
}  
var mediaSource =  
MediaSource.CreateFromAdaptiveMediaSource(result.MediaSource);  
var mediaPlaybackItem = new MediaPlaybackItem(mediaSource);
```

Register for the ID3 tag events using the [MediaPlaybackItem](#) object created in the previous step. This example uses a helper method,

[RegisterMetadataHandlerForID3Cues](#), to register for the events. A lambda expression is used to implement a handler for the [TimedMetadataTracksChanged](#) event, which occurs when the system detects a change in the metadata tracks associated with a [MediaPlaybackItem](#). In some cases, the metadata tracks may be available when the playback item is initially resolved, so outside of the [TimedMetadataTracksChanged](#) handler, we also loop through the available metadata tracks and call [RegisterMetadataHandlerForID3Cues](#).

C#

```
AdaptiveMediaSourceCreationResult result =  
    await AdaptiveMediaSource.CreateFromUriAsync(new  
Uri("http://contoso.com/playlist.m3u"));  
  
if (result.Status != AdaptiveMediaSourceCreationStatus.Success)  
{  
    // TODO: Handle adaptive media source creation errors.  
    return;  
}  
var mediaSource =  
MediaSource.CreateFromAdaptiveMediaSource(result.MediaSource);  
var mediaPlaybackItem = new MediaPlaybackItem(mediaSource);
```

After registering for the ID3 metadata events, the [MediaItem](#) is assigned to a [MediaPlayer](#) for playback within a [MediaPlayerElement](#).

C#

```
_mediaPlayer = new MediaPlayer();
mediaPlayerElement.SetMediaPlayer(_mediaPlayer);
_mediaPlayer.Source = mediaPlaybackItem;
_mediaPlayer.Play();
```

In the **RegisterMetadataHandlerForID3Cues** helper method, get an instance of the **TimedMetadataTrack** class by indexing into the **TimedMetadataTracks** collection of the **MediaPlaybackItem**. Check the **DispatchType** property of the metadata track, which will have a value containing the GUID string "15260DFFFF49443320FF49443320000F" if the track represents ID3 tags. Register for the **CueEntered** event and the **CueExited** event. Then, you must call **SetPresentationMode** on the playback item's **TimedMetadataTracks** collection, to instruct the system that the app wants to receive metadata cue events for this playback item.

C#

```
private void RegisterMetadataHandlerForID3Cues(MediaPlaybackItem item, int
index)
{
    var timedTrack = item.TimedMetadataTracks[index];
    var dispatchType = timedTrack.DispatchType;

    if (String.Equals(dispatchType, "15260DFFFF49443320FF49443320000F",
StringComparison.OrdinalIgnoreCase))
    {
        timedTrack.Label = "ID3 tags";
        timedTrack.CueEntered += metadata_ID3CueEntered;
        timedTrack.CueExited += metadata_ID3CueExited;
        item.TimedMetadataTracks.SetPresentationMode((uint)index,
TimedMetadataTrackPresentationMode.ApplicationPresented);
    }
}
```

In the handler for the **CueEntered** event, cast the data cue contained in the **Cue** property of the **MediaCueEventArgs** to an **DataCue**. Check to make sure the **DataCue** and the **Data** property of the cue are not null. Extended EMU comments are provided in the form raw bytes in the transport stream (see [ID3](#)). Create a new **DataReader** to read the cue data by calling **DataReader.FromBuffer**. In this example, the header values from the ID3 tag are read from the cue data and written to the debug output.

C#

```
private void metadata_ID3CueEntered(TimedMetadataTrack timedMetadataTrack,
MediaCueEventArgs args)
{
```

```

var dataCue = args.Cue as DataCue;
if (dataCue != null && dataCue.Data != null)
{
    // The payload is the raw ID3 bytes found in a TS stream
    // Ref: http://id3.org/id3v2.4.0-structure
    var dr =
Windows.Storage.Streams.DataReader.FromBuffer(dataCue.Data);
    var header_ID3 = dr.ReadString(3);
    var header_version_major = dr.ReadByte();
    var header_version_minor = dr.ReadByte();
    var header_flags = dr.ReadByte();
    var header_tagSize = dr.ReadUInt32();

    System.Diagnostics.Debug.WriteLine($"ID3 tag data: major
{header_version_major}, minor: {header_version_minor}");
}
}

```

Fragmented mp4 emsg boxes

Starting with Windows 10, version 1703, UWP apps can register for cues that correspond to emsg boxes within fragmented mp4 streams. An example usage of this type of metadata is for content providers to signal client applications to play an ad during live streaming content. This example uses [AdaptiveMediaSource](#) to play the media content. For more information, see [Adaptive Streaming](#). Create an [AdaptiveMediaSource](#) for the content by calling [CreateFromUriAsync](#) or [CreateFromStreamAsync](#). Create a [MediaSource](#) object by calling [CreateFromAdaptiveMediaSource](#) and then create a [MediaPlaybackItem](#) from the [MediaSource](#).

C#

```

AdaptiveMediaSourceCreationResult result =
    await AdaptiveMediaSource.CreateFromUriAsync(new
Uri("http://contoso.com/playlist.m3u"));

if (result.Status != AdaptiveMediaSourceCreationStatus.Success)
{
    // TODO: Handle adaptive media source creation errors.
    return;
}
var mediaSource =
MediaSource.CreateFromAdaptiveMediaSource(result.MediaSource);
var mediaPlaybackItem = new MediaPlaybackItem(mediaSource);

```

Register for the emsg box events using the [MediaPlaybackItem](#) object created in the previous step. This example uses a helper method, [RegisterMetadataHandlerForEmsgCues](#), to register for the events. A lambda expression

is used to implement a handler for the [TimedMetadataTracksChanged](#) event, which occurs when the system detects a change in the metadata tracks associated with a **MediaPlaybackItem**. In some cases, the metadata tracks may be available when the playback item is initially resolved, so outside of the **TimedMetadataTracksChanged** handler, we also loop through the available metadata tracks and call **RegisterMetadataHandlerForEmsgCues**.

C#

```
AdaptiveMediaSourceCreationResult result =  
    await AdaptiveMediaSource.CreateFromUriAsync(new  
Uri("http://contoso.com/playlist.m3u"));  
  
if (result.Status != AdaptiveMediaSourceCreationStatus.Success)  
{  
    // TODO: Handle adaptive media source creation errors.  
    return;  
}  
  
var mediaSource =  
MediaSource.CreateFromAdaptiveMediaSource(result.MediaSource);  
var mediaPlaybackItem = new MediaPlaybackItem(mediaSource);
```

After registering for the emsg box metadata events, the **MediaItem** is assigned to a **MediaPlayer** for playback within a **MediaPlayerElement**.

C#

```
_mediaPlayer = new MediaPlayer();  
mediaPlayerElement.SetMediaPlayer(_mediaPlayer);  
_mediaPlayer.Source = mediaPlaybackItem;  
_mediaPlayer.Play();
```

In the **RegisterMetadataHandlerForEmsgCues** helper method, get an instance of the [TimedMetadataTrack](#) class by indexing into the **TimedMetadataTracks** collection of the **MediaPlaybackItem**. Check the **DispatchType** property of the metadata track, which will have a value of "emsg:mp4" if the track represents emsg boxes. Register for the [CueEntered](#) event and the [CueExited](#) event. Then, you must call [SetPresentationMode](#) on the playback item's **TimedMetadataTracks** collection, to instruct the system that the app wants to receive metadata cue events for this playback item.

C#

```
private void RegisterMetadataHandlerForEmsgCues(MediaPlaybackItem item, int  
index)  
{  
    var timedTrack = item.TimedMetadataTracks[index];  
    var dispatchType = timedTrack.DispatchType;
```



```

        if (String.Equals(dispatchType, "emsg:mp4",
StringComparison.OrdinalIgnoreCase))
        {
            timedTrack.Label = "mp4 Emsg boxes";
            timedTrack.CueEntered += metadata_EmsgCueEntered;
            timedTrack.CueExited += metadata_EmsgCueExited;
            item.TimedMetadataTracks.SetPresentationMode((uint)index,
TimedMetadataTrackPresentationMode.ApplicationPresented);
        }
    }
}

```

In the handler for the **CueEntered** event, cast the data cue contained in the **Cue** property of the **MediaCueEventArgs** to an **DataCue**. Check to make sure the **DataCue** object is not null. The properties of the emsg box are provided by the media pipeline as custom properties in the **DataCue** object's **Properties** collection. This example attempts to extract several different property values using the **TryGetValue** method. If this method returns null, it means the requested property is not present in the emsg box, so a default value is set instead.

The next part of the example illustrates the scenario where ad playback is triggered, which is the case when the *scheme_id_uri* property, obtained in the previous step, has a value of "urn:scte:scte35:2013:xml". For more information, see https://dashif.org/identifiers/event_schemes/. Note that the standard recommends sending this emsg multiple times for redundancy, so this example maintains a list of the emsg IDs that have already been processed and only processes new messages. Create a new **DataReader** to read the cue data by calling **DataReader.FromBuffer** and set the encoding to UTF-8 by setting the **UnicodeEncoding** property, then read the data. In this example, the message payload is written to the debug output. A real app would use the payload data to schedule the playback of an ad.

C#

```

private void metadata_EmsgCueEntered(TimedMetadataTrack timedMetadataTrack,
MediaCueEventArgs args)
{
    var dataCue = args.Cue as DataCue;
    if (dataCue != null)
    {
        string scheme_id_uri = string.Empty;
        string value = string.Empty;
        UInt32 timescale = (UInt32)TimeSpan.TicksPerSecond;
        UInt32 presentation_time_delta = (UInt32)dataCue.StartTime.Ticks;
        UInt32 event_duration = (UInt32)dataCue.Duration.Ticks;
        UInt32 id = 0;
        Byte[] message_data = null;

        const string scheme_id_uri_key = "emsg:scheme_id_uri";
    }
}

```

```

object propValue = null;
dataCue.Properties.TryGetValue(scheme_id_uri_key, out propValue);
scheme_id_uri = propValue != null ? (string)propValue : "";

const string value_key = "emsg:value";
propValue = null;
dataCue.Properties.TryGetValue(value_key, out propValue);
value = propValue != null ? (string)propValue : "";

const string timescale_key = "emsg:timescale";
propValue = null;
dataCue.Properties.TryGetValue(timescale_key, out propValue);
timescale = propValue != null ? (UInt32)propValue : timescale;

const string presentation_time_delta_key =
"emsg:presentation_time_delta";
propValue = null;
dataCue.Properties.TryGetValue(presentation_time_delta_key, out
propValue);
presentation_time_delta = propValue != null ? (UInt32)propValue :
presentation_time_delta;

const string event_duration_key = "emsg:event_duration";
propValue = null;
dataCue.Properties.TryGetValue(event_duration_key, out propValue);
event_duration = propValue != null ? (UInt32)propValue :
event_duration;

const string id_key = "emsg:id";
propValue = null;
dataCue.Properties.TryGetValue(id_key, out propValue);
id = propValue != null ? (UInt32)propValue : 0;

System.Diagnostics.Debug.WriteLine($"Label:
{timedMetadataTrack.Label}, Id: {dataCue.Id}, StartTime:
{dataCue.StartTime}, Duration: {dataCue.Duration}");
System.Diagnostics.Debug.WriteLine($"scheme_id_uri: {scheme_id_uri},
value: {value}, timescale: {timescale}, presentation_time_delta:
{presentation_time_delta}, event_duration: {event_duration}, id: {id}");

if (dataCue.Data != null)
{
    var dr =
Windows.Storage.Streams.DataReader.FromBuffer(dataCue.Data);

    // Check if this is a SCTE ad message:
    // Ref: http://dashif.org/identifiers/event-schemes/
    if (scheme_id_uri.ToLower() == "urn:scte:scte35:2013:xml")
    {
        // SCTE recommends publishing emsg more than once, so we
        avoid reprocessing the same message id:
        if (!processedAdIds.Contains(id))
        {
            processedAdIds.Add(id);
            dr.UnicodeEncoding =

```

```

Windows.Storage.Streams.UnicodeEncoding.Utf8;
        var scte35payload = dr.ReadString(dataCue.Data.Length);
        System.Diagnostics.Debug.WriteLine($"", message_data:
{scte35payload}");
        // TODO: ScheduleAdFromScte35Payload(timedMetadataTrack,
presentation_time_delta, timescale, event_duration, scte35payload);
    }
    else
    {
        System.Diagnostics.Debug.WriteLine($"This emsg.Id, {id},
has already been processed.");
    }
}
else
{
    message_data = new byte[dataCue.Data.Length];
    dr.ReadBytes(message_data);
    // TODO: Use the 'emsg' bytes for something useful.
    System.Diagnostics.Debug.WriteLine($"", message_data.Length:
{message_data.Length}");
}
}
}
}

```

Related topics

- [Media playback](#)
- [Media items, playlists, and tracks](#)

Feedback

Was this page helpful?

 Yes

 No

[Provide product feedback](#)  | [Get help at Microsoft Q&A](#)

Create, schedule, and manage media breaks

Article • 10/20/2022

This article shows you how to create, schedule, and manage media breaks to your media playback app. Media breaks are typically used to insert audio or video ads into media content. Starting with Windows 10, version 1607, you can use the [MediaBreakManager](#) class to quickly and easily add media breaks to any [MediaPlaybackItem](#) that you play with a [MediaPlayer](#).

After you schedule one or more media breaks, the system will automatically play your media content at the specified time during playback. The **MediaBreakManager** provides events so that your app can react when media breaks start, end, or when they are skipped by the user. You can also access a [MediaPlaybackSession](#) for your media breaks to monitor events such as download and buffering progress updates.

Schedule media breaks

Every **MediaPlaybackItem** object has its own [MediaBreakSchedule](#) that you use to configure the media breaks that will play when the item is played. The first step for using media breaks in your app is to create a [MediaPlaybackItem](#) for your main playback content.

C#

```
MediaPlaybackItem moviePlaybackItem =  
    new MediaPlaybackItem(MediaSource.CreateFromUri(new  
        Uri("http://www.fabrikam.com/movie.mkv"))));
```

For more information about working with **MediaPlaybackItem**, [MediaPlaybackList](#) and other fundamental media playback APIs, see [Media items, playlists, and tracks](#).

The next example shows how to add a preroll break to the **MediaPlaybackItem**, which means that the system will play the media break before playing the playback item to which the break belongs. First a new [MediaBreak](#) object is instantiated. In this example, the constructor is called with [MediaBreakInsertionMethod.Interrupt](#), meaning that the main content will be paused while the break content is played.

Next, a new **MediaPlaybackItem** is created for the content that will be played during the break, such as an ad. The [CanSkip](#) property of this playback item is set to false. This

means that the user will not be able to skip the item using the built-in media controls. Your app can still choose to skip the ad programmatically by calling [SkipCurrentBreak](#).

The media break's [PlaybackList](#) property is a [MediaPlaybackList](#) that allows you to play multiple media items as a playlist. Add one or more [MediaPlaybackItem](#) objects from the list's **Items** collection to include them in the media break's playlist.

Finally, schedule the media break by using the main content playback item's [BreakSchedule](#) property. Specify the break to be a preroll break by assigning it to the [PrerollBreak](#) property of the schedule object.

C#

```
MediaBreak preRollMediaBreak = new
MediaBreak(MediaBreakInsertionMethod.Interrupt);
MediaPlaybackItem prerollAd =
    new MediaPlaybackItem(MediaSource.CreateFromUri(new
Uri("http://www.fabrikam.com/preroll_ad.mp4")));
prerollAd.CanSkip = false;
preRollMediaBreak.PlaybackList.Items.Add(prerollAd);

moviePlaybackItem.BreakSchedule.PrerollBreak = preRollMediaBreak;
```

Now you can play back the main media item, and the media break that you created will play before the main content. Create a new [MediaPlayer](#) object and optionally set the [AutoPlay](#) property to true to start playback automatically. Set the [Source](#) property of the [MediaPlayer](#) to your main content playback item. It's not required, but you can assign the [MediaPlayer](#) to a [MediaPlayerElement](#) to render the media in a XAML page. For more information about using [MediaPlayer](#), see [Play audio and video with MediaPlayer](#).

C#

```
_mediaPlayer = new MediaPlayer();
_mediaPlayer.AutoPlay = true;
_mediaPlayer.Source = moviePlaybackItem;
mediaPlayerElement.SetMediaPlayer(_mediaPlayer);
```

Add a postroll break that plays after the [MediaPlaybackItem](#) containing your main content finishes playing, by using the same technique as a preroll break, except that you assign your [MediaBreak](#) object to the [PostrollBreak](#) property.

C#

```
MediaBreak postrollMediaBreak = new
MediaBreak(MediaBreakInsertionMethod.Interrupt);
```

```

MediaPlayerItem postRollAd =
    new MediaPlayerItem(MediaSource.CreateFromUri(new
Uri("http://www.fabrikam.com/postroll_ad.mp4")));
postrollMediaBreak.PlaybackList.Items.Add(postRollAd);

moviePlaybackItem.BreakSchedule.PostrollBreak = postrollMediaBreak;

```

You can also schedule one or more midroll breaks that play at a specified time within the playback of the main content. In the following example, the [MediaBreak](#) is created with the constructor overload that accepts a **TimeSpan** object, which specifies the time within the playback of the main media item when the break will be played. Again, [MediaBreakInsertionMethod.Interrupt](#) is specified to indicate that the main content's playback will be paused while the break plays. The midroll break is added to the schedule by calling [InsertMidrollBreak](#). You can get a read-only list of the current midroll breaks in the schedule by accessing the [MidrollBreaks](#) property.

C#

```

MediaBreak midrollMediaBreak = new
MediaBreak(MediaBreakInsertionMethod.Interrupt, TimeSpan.FromMinutes(10));
midrollMediaBreak.PlaybackList.Items.Add(
    new MediaPlayerItem(MediaSource.CreateFromUri(new
Uri("http://www.fabrikam.com/midroll_ad_1.mp4"))));
midrollMediaBreak.PlaybackList.Items.Add(
    new MediaPlayerItem(MediaSource.CreateFromUri(new
Uri("http://www.fabrikam.com/midroll_ad_2.mp4"))));
moviePlaybackItem.BreakSchedule.InsertMidrollBreak(midrollMediaBreak);

```

The next midroll break example shown uses the [MediaBreakInsertionMethod.Replace](#) insertion method, which means that the system will continue processing the main content while the break is playing. This option is typically used by live streaming media apps where you don't want the content to pause and fall behind the live stream while the ad is played.

This example also uses an overload of the [MediaPlayerItem](#) constructor that accepts two [TimeSpan](#) parameters. The first parameter specifies the starting point within the media break item where playback will begin. The second parameter specifies the duration for which the media break item will be played. So, in the following example, the **MediaBreak** will begin playing at 20 minutes into the main content. When it plays, the media item will start 30 seconds from the beginning of the break media item and will play for 15 seconds before the main media content resumes playing.

C#

```

midrollMediaBreak = new MediaBreak(MediaBreakInsertionMethod.Replace,
TimeSpan.FromMinutes(20));

```

```
MediaPlaybackItem ad =  
    new MediaPlaybackItem(MediaSource.CreateFromUri(new  
Uri("http://www.fabrikam.com/midroll_ad_3.mp4")),  
    TimeSpan.FromSeconds(30),  
    TimeSpan.FromSeconds(15));  
ad.CanSkip = false;  
midrollMediaBreak.PlaybackList.Items.Add(ad);
```

Skip media breaks

As mentioned previously in this article, the [CanSkip](#) property of a **MediaPlaybackItem** can be set to prevent the user from skipping the content with the built-in controls. However, you can call [SkipCurrentBreak](#) from your code at any time to skip the current break.

C#

```
private void SkipButton_Click(object sender, RoutedEventArgs e) =>  
    _mediaPlayer.BreakManager.SkipCurrentBreak();
```

Handle MediaBreak events

There are several events related to media breaks that you can register for in order to take action based on the changing state of media breaks.

C#

```
_mediaPlayer.BreakManager.BreakStarted += BreakManager_BreakStarted;  
_mediaPlayer.BreakManager.BreakEnded += BreakManager_BreakEnded;  
_mediaPlayer.BreakManager.BreakSkipped += BreakManager_BreakSkipped;  
_mediaPlayer.BreakManager.BreaksSeekedOver += BreakManager_BreaksSeekedOver;
```

The [BreakStarted](#) is raised when a media break starts. You may want to update your UI to let the user know that media break content is playing. This example uses the [MediaBreakStartedEventArgs](#) passed into the handler to get a reference to the media break that started. Then the [CurrentItemIndex](#) property is used to determine which media item in the media break's playlist is being played. Then the UI is updated to show the user the current ad index and the number of ads remaining in the break. Remember that updates to the UI must be made on the UI thread, so the call should be made inside a call to [RunAsync](#).

C#

```
private async void BreakManager_BreakStarted(MediaBreakManager sender,
MediaBreakStartedEventArgs args)
{
    MediaBreak currentBreak = sender.CurrentBreak;
    var currentIndex = currentBreak.PlaybackList.CurrentItemIndex;
    var itemCount = currentBreak.PlaybackList.Items.Count;

    await Dispatcher.RunAsync(Windows.UI.Core.CoreDispatcherPriority.Normal,
    () =>
        statusTextBlock.Text = $"Playing ad {currentIndex + 1} of
{itemCount}");
}
```

BreakEnded is raised when all of the media items in the break have finished playing or have been skipped over. You can use the handler for this event to update the UI to indicate that media break content is no longer playing.

C#

```
private async void BreakManager_BreakEnded(MediaBreakManager sender,
MediaBreakEndedEventArgs args)
{
    // Update UI to show that the MediaBreak is no longer playing
    await Dispatcher.RunAsync(Windows.UI.Core.CoreDispatcherPriority.Normal,
    () => statusTextBlock.Text = "");

    args.MediaBreak.CanStart = false;
}
```

The **BreakSkipped** event is raised when the user presses the *Next* button in the built-in UI during playback of an item for which **CanSkip** is true, or when you skip a break in your code by calling **SkipCurrentBreak**.

The following example uses the **Source** property of the **MediaPlayer** to get a reference to the media item for the main content. The skipped media break belongs to the break schedule of this item. Next, the code checks to see if the media break that was skipped is the same as the media break set to the **PrerollBreak** property of the schedule. If so, this means that the preroll break was the break that was skipped, and in this case, a new midroll break is created and scheduled to play 10 minutes into the main content.

C#

```
private async void BreakManager_BreakSkipped(MediaBreakManager sender,
MediaBreakSkippedEventArgs args)
{
    // Update UI to show that the MediaBreak is no longer playing
    await Dispatcher.RunAsync(Windows.UI.Core.CoreDispatcherPriority.Normal,
```



```

() => statusTextBlock.Text = "");

    MediaPlayerItem currentItem = _mediaPlayer.Source as
MediaPlayerItem;
    if(!(currentItem.BreakSchedule.PrerollBreak is null)
        && currentItem.BreakSchedule.PrerollBreak == args.MediaBreak)
    {
        MediaBreak mediaBreak = new
MediaBreak(MediaBreakInsertionMethod.Interrupt, TimeSpan.FromMinutes(10));
        mediaBreak.PlaybackList.Items.Add(await GetAdPlaybackItem());
        currentItem.BreakSchedule.InsertMidrollBreak(mediaBreak);
    }
}

```

BreaksSearchedOver is raised when the playback position of the main media item passes over the scheduled time for one or more media breaks. The following example checks to see if more than one media break was sought over, if the playback position was moved forward, and if it was moved forward less than 10 minutes. If so, the first break that was sought over, obtained from the **SearchedOverBreaks** collection exposed by the event args, is played immediately with a call to the **PlayBreak** method of the **MediaPlayer.BreakManager**.

```

C#

private void BreakManager_BreaksSearchedOver(MediaBreakManager sender,
MediaBreakSearchedOverEventArgs args)
{
    if(args.SearchedOverBreaks.Count > 1
        && args.NewPosition.TotalMinutes > args.OldPosition.TotalMinutes
        && args.NewPosition.TotalMinutes - args.OldPosition.TotalMinutes <
10.0)
        _mediaPlayer.BreakManager.PlayBreak(args.SearchedOverBreaks[0]);
}

```

Access the current playback session

The **MediaPlayerSession** object uses the **MediaPlayer** class to provide data and events related to the currently playing media content. The **MediaBreakManager** also has a **MediaPlayerSession** that you can access to get data and events specifically related to the media break content that is being played. Information you can get from the playback session includes the current playback state, playing or paused, and the current playback position within the content. You can use the **NaturalVideoWidth** and **NaturalVideoHeight** properties and the **NaturalVideoSizeChanged** to adjust your video UI if the media break content has a different aspect ratio than your main content. You can also receive events such as **BufferingStarted**, **BufferingEnded**, and

[DownloadProgressChanged](#) that can provide valuable telemetry about the performance of your app.

The following example registers a handler for the **BufferingProgressChanged** event; in the event handler, it updates the UI to show the current buffering progress.

C#

```
_mediaPlayer.BreakManager.PlaybackSession.BufferingProgressChanged +=  
PlaybackSession_BufferingProgressChanged;
```

C#

```
private async void  
PlaybackSession_BufferingProgressChanged(MediaPlaybackSession sender, object  
args)  
{  
    await Dispatcher.RunAsync(Windows.UI.Core.CoreDispatcherPriority.Normal,  
    () =>  
        bufferingProgressBar.Value = sender.BufferingProgress);  
}
```

Related topics

- [Media playback](#)
- [Play audio and video with MediaPlayer](#)
- [Manual control of the System Media Transport Controls](#)

Feedback

Was this page helpful?

 Yes

 No

[Provide product feedback](#)  | [Get help at Microsoft Q&A](#)

Play media in the background

Article • 10/20/2022

This article shows you how to configure your app so that media continues to play when your app moves from the foreground to the background. This means that even after the user has minimized your app, returned to the home screen, or has navigated away from your app in some other way, your app can continue to play audio.

Scenarios for background audio playback include:

- **Long-running playlists:** The user briefly brings up a foreground app to select and start a playlist, after which the user expects the playlist to continue playing in the background.
- **Using task switcher:** The user briefly brings up a foreground app to start playing audio, then switches to another open app using the task switcher. The user expects the audio to continue playing in the background.

The background audio implementation described in this article will allow your app to run universally on all Windows devices including Mobile, Desktop, and Xbox.

📌 Note

The code in this article was adapted from the [UWP Background Audio sample](#).

Explanation of one-process model

With Windows 10, version 1607, a new single-process model has been introduced that greatly simplifies the process of enabling background audio. Previously, your app was required to manage a background process in addition to your foreground app and then manually communicate state changes between the two processes. Under the new model, you simply add the background audio capability to your app manifest, and your app will automatically continue playing audio when it moves to the background. Two new application lifecycle events, [EnteredBackground](#) and [LeavingBackground](#) let your app know when it is entering and leaving the background. When your app moves into the transitions to or from the background, the memory constraints enforced by the system may change, so you can use these events to check your current memory consumption and free up resources in order to stay below the limit.

By eliminating the complex cross-process communication and state management, the new model allows you to implement background audio much more quickly with a significant reduction in code. However, the two-process model is still supported in the current release for backwards compatibility. For more information, see [Legacy background audio model](#).

Requirements for background audio

Your app must meet the following requirements for audio playback while your app is in the background.

- Add the **Background Media Playback** capability to your app manifest, as described later in this article.
- If your app disables the automatic integration of **MediaPlayer** with the System Media Transport Controls (SMTC), such as by setting the [CommandManager.IsEnabled](#) property to false, then you must implement manual integration with the SMTC in order to enable background media playback. You must also manually integrate with SMTC if you are using an API other than **MediaPlayer**, such as [AudioGraph](#), to play audio if you want to have the audio continue to play when your app moves to the background. The minimum SMTC integration requirements are described in the "Use the system media transport controls for background audio" section of [Manual control of the System Media Transport Controls](#).
- While your app is in the background, you must stay under the memory usage limits set by the system for background apps. Guidance for managing memory while in the background is provided later in this article.

Background media playback manifest capability

To enable background audio, you must add the background media playback capability to the app manifest file, `Package.appxmanifest`.

To add capabilities to the app manifest using the manifest designer

1. In Microsoft Visual Studio, in **Solution Explorer**, open the designer for the application manifest by double-clicking the `package.appxmanifest` item.
2. Select the **Capabilities** tab.
3. Select the **Background Media Playback** check box.

To set the capability by manually editing the app manifest xml, first make sure that the `uap3` namespace prefix is defined in the **Package** element. If not, add it as shown below.

XML

```
<Package
  xmlns="http://schemas.microsoft.com/appx/manifest/foundation/windows10"
  xmlns:mp="http://schemas.microsoft.com/appx/2014/phone/manifest"
  xmlns:uap="http://schemas.microsoft.com/appx/manifest/uap/windows10"
  xmlns:uap3="http://schemas.microsoft.com/appx/manifest/uap/windows10/3"
  IgnorableNamespaces="uap uap3 mp">
```

Next, add the *backgroundMediaPlayback* capability to the **Capabilities** element:

XML

```
<Capabilities>
  <uap3:Capability Name="backgroundMediaPlayback"/>
</Capabilities>
```

Handle transitioning between foreground and background

When your app moves from the foreground to the background, the [EnteredBackground](#) event is raised. And when your app returns to the foreground, the [LeavingBackground](#) event is raised. Because these are app lifecycle events, you should register handlers for these events when your app is created. In the default project template, this means adding it to the **App** class constructor in *App.xaml.cs*.

C#

```
public App()
{
    this.InitializeComponent();
    this.Suspending += OnSuspending;

    this.EnteredBackground += App_EnteredBackground;
    this.LeavingBackground += App_LeavingBackground;
}
```

Create a variable to track whether you are currently running in the background.

C#

```
bool _isInBackgroundMode = false;
```

When the [EnteredBackground](#) event is raised, set the tracking variable to indicate that you are currently running in the background. You should not perform long-running tasks in the **EnteredBackground** event because this may cause the transition to the background to appear slow to the user.

C#

```
private void App_EnteredBackground(object sender, EnteredBackgroundEventArgs e)
{
    _isInBackgroundMode = true;
}
```

In the [LeavingBackground](#) event handler, you should set the tracking variable to indicate that your app is no longer running in the background.

C#

```
private void App_LeavingBackground(object sender, LeavingBackgroundEventArgs e)
{
    _isInBackgroundMode = false;
}
```

Memory management requirements

The most important part of handling the transition between foreground and background is managing the memory that your app uses. Because running in the background will reduce the memory resources your app is allowed to retain by the system, you should also register for the [AppMemoryUsageIncreased](#) and [AppMemoryUsageLimitChanging](#) events. When these events are raised, you should check your app's current memory usage and the current limit, and then reduce your memory usage if needed. For information about reducing your memory usage while running in the background, see [Free memory when your app moves to the background](#).

Network availability for background media apps

All network aware media sources, those that are not created from a stream or a file, will keep the network connection active while retrieving remote content, and they release it when they are not. [MediaStreamSource](#), specifically, relies on the application to correctly report the correct buffered range to the platform using [SetBufferedRange](#).

After the entire content is fully buffered, the network will no longer be reserved on the app's behalf.

If you need to make network calls that occur in the background when media is not downloading, these must be wrapped in an appropriate task like [MaintenanceTrigger](#) or [TimeTrigger](#). For more information, see [Support your app with background tasks](#).

Related topics

- [Media playback](#)
- [Play audio and video with MediaPlayer](#)
- [Integrate with the System Media Transport Controls](#)
- [Background Audio sample](#) 

Legacy background media playback

Article • 10/20/2022

This article describes the legacy, two-process model for adding background audio support to your UWP app. Starting with Windows 10, version 1607, a single-process model for background audio that is much simpler to implement. For more information on the current recommendations for background audio, see [Play media in the background](#). This article is intended to provide support for apps that have already been developed using the legacy two-process model.

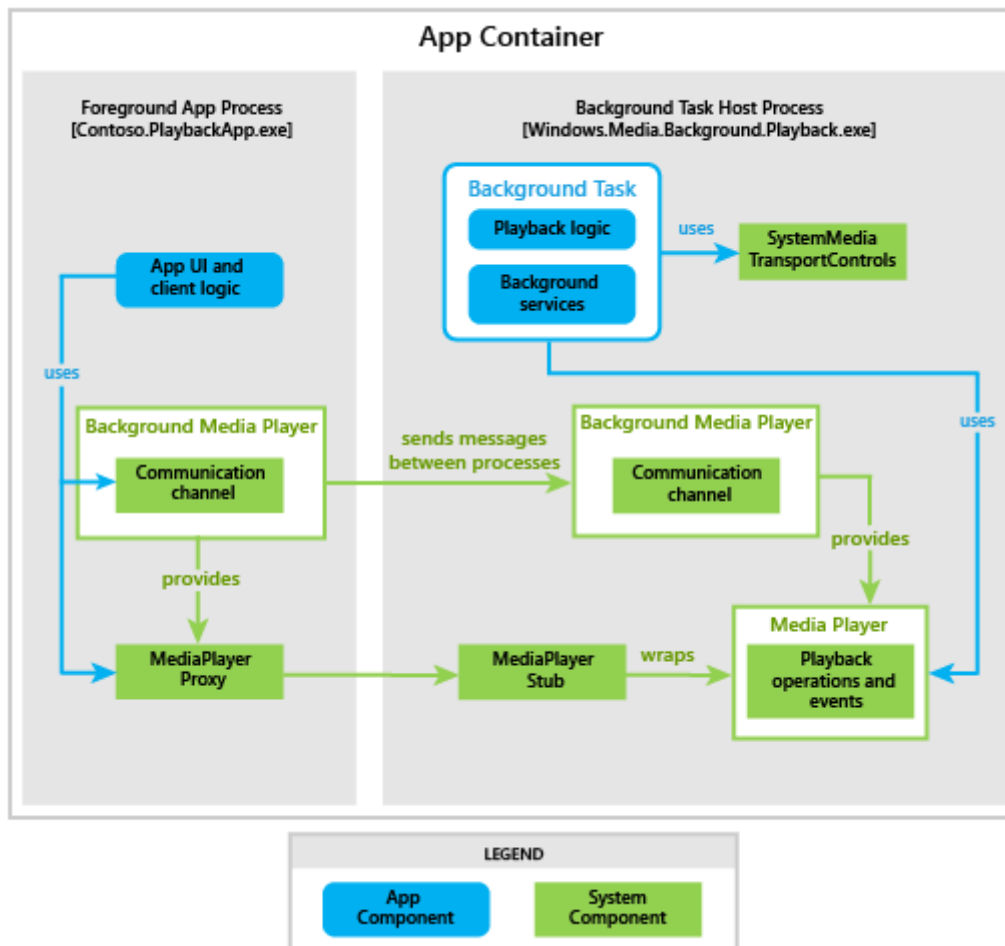
ⓘ Note

Starting with Windows, version 1703, **BackgroundMediaPlayer** is deprecated and may not be available in future versions of Windows.

Background audio architecture

An app performing background playback consists of two processes. The first process is the main app, which contains the app UI and client logic, running in the foreground. The second process is the background playback task, which implements [IBackgroundTask](#) like all UWP app background tasks. The background task contains the audio playback logic and background services. The background task communicates with the system through the System Media Transport Controls.

The following diagram is an overview of how the system is designed.



MediaPlayer

The [Windows.Media.Playback](#) namespace contains APIs used to play audio in the background. There is a single instance of [MediaPlayer](#) per app through which playback occurs. Your background audio app calls methods and sets properties on the **MediaPlayer** class to set the current track, start playback, pause, fast forward, rewind, and so on. The media player object instance is always accessed through the [BackgroundMediaPlayer.Current](#) property.

MediaPlayer Proxy and Stub

When **BackgroundMediaPlayer.Current** is accessed from your app's background process, the **MediaPlayer** instance is activated in the background task host and can be manipulated directly.

When **BackgroundMediaPlayer.Current** is accessed from the foreground application, the **MediaPlayer** instance that is returned is actually a proxy that communicates with a stub in the background process. This stub communicates with the actual **MediaPlayer** instance, which is also hosted in the background process.

Both the foreground and background process can access most of the properties of the **MediaPlayer** instance, with the exception of **MediaPlayer.Source** and **MediaPlayer.SystemMediaTransportControls** which can only be accessed from the background process. The foreground app and the background process can both receive notifications of media-specific events like **MediaOpened**, **MediaEnded**, and **MediaFailed**.

Playback Lists

A common scenario for background audio applications is to play multiple items in a row. This is most easily accomplished in your background process by using a **MediaPlaybackList** object, which can be set as a source on the **MediaPlayer** by assigning it to the **MediaPlayer.Source** property.

It is not possible to access a **MediaPlaybackList** from the foreground process that was set in the background process.

System Media Transport Controls

A user may control audio playback without directly using your app's UI through means such as Bluetooth devices, SmartGlass, and the System Media Transport Controls. Your background task uses the **SystemMediaTransportControls** class to subscribe to these user-initiated system events.

To get a **SystemMediaTransportControls** instance from within the background process, use the **MediaPlayer.SystemMediaTransportControls** property. Foreground apps get an instance of the class by calling **SystemMediaTransportControls.GetForCurrentView**, but the instance returned is a foreground-only instance that does not relate to the background task.

Sending Messages Between Tasks

There are times when you will want to communicate between the two processes of a background audio app. For example, you might want the background task to notify the foreground task when a new track starts playing, and then send the new song title to the foreground task to display on the screen.

A simple communication mechanism raises events in both the foreground and background processes. The **SendMessageToForeground** and **SendMessageToBackground** methods each invoke events in the corresponding process.

Messages can be received by subscribing to the [MessageReceivedFromBackground](#) and [MessageReceivedFromForeground](#) events.

Data can be passed as an argument to the send message methods that are then passed into the message received event handlers. Pass data using the [ValueSet](#) class. This class is a dictionary that contains a string as a key and other value types as values. You can pass simple value types such as integers, strings, and booleans.

Background Task Life Cycle

The lifetime of a background task is closely tied to your app's current playback status. For example, when the user pauses audio playback, the system may terminate or cancel your app depending on the circumstances. After a period of time without audio playback, the system may automatically shut down the background task.

The [IBackgroundTask.Run](#) method is called the first time your app accesses either [BackgroundMediaPlayer.Current](#) from code running in the foreground app or when you register a handler for the [MessageReceivedFromBackground](#) event, whichever occurs first. It is recommended that you register for the message received handler before calling [BackgroundMediaPlayer.Current](#) for the first time so that the foreground app doesn't miss any messages sent from the background process.

To keep the background task alive, your app must request a [BackgroundTaskDeferral](#) from within the [Run](#) method and call [BackgroundTaskDeferral.Complete](#) when the task instance receives the [Canceled](#) or [Completed](#) events. Do not loop or wait in the [Run](#) method because this consumes resources and may cause your app's background task to be terminated by the system.

Your background task gets the [Completed](#) event when the [Run](#) method is completed and deferral is not requested. In some cases, when your app gets the [Canceled](#) event, it can be also followed by the [Completed](#) event. Your task may receive a [Canceled](#) event while [Run](#) is executing, so be sure to manage this potential concurrence.

Situations in which the background task can be cancelled include:

- A new app with audio playback capabilities starts on systems that enforce the exclusivity sub-policy. See the [System policies for background audio task lifetime](#) section below.
- A background task has been launched but music is not yet playing, and then the foreground app is suspended.
- Other media interruptions, such as incoming phone calls or VoIP calls.

Situations in which the background task can be terminated without notice include:

- A VoIP call comes in and there is not enough available memory on the system to keep the background task alive.
- A resource policy is violated.
- Task cancellation or completion does not end gracefully.

System policies for background audio task lifetime

The following policies help determine how the system manages the lifetime of background audio tasks.

Exclusivity

If enabled, this sub-policy limits the number of background audio tasks to be at most 1 at any given time. It is enabled on Mobile and other non-Desktop SKUs.

Inactivity Timeout

Due to resource constraints, the system may terminate your background task after a period of inactivity.

A background task is considered “inactive” if both of the following conditions are met:

- The foreground app is not visible (it is suspended or terminated).
- The background media player is not in the playing state.

If both of these conditions are satisfied, the background media system policy will start a timer. If neither condition has changed when the timer expires, the background media system policy will terminate the background task.

Shared Lifetime

If enabled, this sub-policy forces the background task to be dependent on the lifetime of the foreground task. If the foreground task is shut down, either by the user or the system, the background task will also shut down.

However, note that this does not mean that the foreground is dependent on the background. If the background task is shut down, this does not force the foreground task to shut down.

The following table lists the which policies are enforced on which device types.

Sub-policy	Desktop	Mobile	Other
Exclusivity	Disabled	Enabled	Enabled
Inactivity Timeout	Disabled	Enabled	Disabled
Shared Lifetime	Enabled	Disabled	Disabled

Adaptive streaming

Article • 10/20/2022

This article describes how to add playback of adaptive streaming multimedia content to a Universal Windows Platform (UWP) app. This feature supports playback of Http Live Streaming (HLS) and Dynamic Streaming over HTTP (DASH) content.

Starting with Windows 10, version 1803, Smooth Streaming is supported by [AdaptiveMediaSource](#). Note that for Smooth Streaming, only H264 and WVC1 codecs are supported. Other manifest types do not have this limitation.

For a list of supported HLS protocol tags, see [HLS tag support](#).

For a list of supported DASH profiles, see [DASH profile support](#).

ⓘ Note

The code in this article was adapted from the UWP [Adaptive streaming sample](#) [↗].

Simple adaptive streaming with MediaPlayer and MediaPlayerElement

To play adaptive streaming media in a UWP app, create a **Uri** object pointing to a DASH or HLS manifest file. Create an instance of the [MediaPlayer](#) class. Call [MediaSource.CreateFromUri](#) to create a new **MediaSource** object and then set that to the [Source](#) property of the **MediaPlayer**. Call [Play](#) to start playback of the media content.

C#

```
MediaPlayer _mediaPlayer;
```

C#

```
System.Uri manifestUri = new  
Uri("http://amssamples.streaming.mediaservices.windows.net/49b57c87-f5f3-  
48b3-ba22-c55cfdffa9cb/Sintel.ism/manifest(format=m3u8-aapl)");  
_mediaPlayer = new MediaPlayer();  
_mediaPlayer.Source = MediaSource.CreateFromUri(manifestUri);  
_mediaPlayer.Play();
```

The above example will play the audio of the media content but it doesn't automatically render the content in your UI. Most apps that play video content will want to render the content in a XAML page. To do this, add a **MediaPlayerElement** control to your XAML page.

XML

```
<MediaPlayerElement x:Name="mediaPlayerElement"
HorizontalAlignment="Stretch" AreTransportControlsEnabled="True"/>
```

Call **MediaSource.CreateFromUri** to create a **MediaSource** from the URI of a DASH or HLS manifest file. Then set the **Source** property of the **MediaPlayerElement**. The **MediaPlayerElement** will automatically create a new **MediaPlayer** object for the content. You can call **Play** on the **MediaPlayer** to start playback of the content.

C#

```
System.Uri manifestUri = new
Uri("http://amssamples.streaming.mediaservices.windows.net/49b57c87-f5f3-
48b3-ba22-c55cfdffa9cb/Sintel.ism/manifest(format=m3u8-aapl)");
mediaPlayerElement.Source = MediaSource.CreateFromUri(manifestUri);
mediaPlayerElement.MediaPlayer.Play();
```

ⓘ Note

Starting with Windows 10, version 1607, it is recommended that you use the **MediaPlayer** class to play media items. The **MediaPlayerElement** is a lightweight XAML control that is used to render the content of a **MediaPlayer** in a XAML page. The **MediaElement** control continues to be supported for backwards compatibility. For more information about using **MediaPlayer** and **MediaPlayerElement** to play media content, see [Play audio and video with MediaPlayer](#). For information about using **MediaSource** and related APIs to work with media content, see [Media items, playlists, and tracks](#).

Adaptive streaming with AdaptiveMediaSource

If your app requires more advanced adaptive streaming features, such as providing custom HTTP headers, monitoring the current download and playback bitrates, or adjusting the ratios that determine when the system switches bitrates of the adaptive stream, use the **AdaptiveMediaSource** object.

The adaptive streaming APIs are found in the [Windows.Media.Streaming.Adaptive](#) namespace. The examples in this article use APIs from the following namespaces.

C#

```
using Windows.Media.Streaming.Adaptive;
using System.Threading.Tasks;
using Windows.Storage.Streams;
using Windows.Media.Playback;
using Windows.Media.Core;
```

Initialize an AdaptiveMediaSource from a URI.

Initialize the **AdaptiveMediaSource** with the URI of an adaptive streaming manifest file by calling [CreateFromUriAsync](#). The [AdaptiveMediaSourceCreationStatus](#) value returned from this method lets you know if the media source was created successfully. If so, you can set the object as the stream source for your **MediaPlayer** by creating a **MediaSource** object by calling [MediaSource.CreateFromAdaptiveMediaSource](#), and then assigning it to the media player's [Source](#) property. In this example, the [AvailableBitrates](#) property is queried to determine the maximum supported bitrate for this stream, and then that value is set as the initial bitrate. This example also registers handlers for the several **AdaptiveMediaSource** events that are discussed later in this article.

C#

```
async private void InitializeAdaptiveMediaSource(System.Uri uri)
{
    AdaptiveMediaSourceCreationResult result = await
    AdaptiveMediaSource.CreateFromUriAsync(uri);

    if (result.Status == AdaptiveMediaSourceCreationStatus.Success)
    {
        ams = result.MediaSource;
        mediaPlayerElement.SetMediaPlayer(new MediaPlayer());
        mediaPlayerElement.MediaPlayer.Source =
        MediaSource.CreateFromAdaptiveMediaSource(ams);
        mediaPlayerElement.MediaPlayer.Play();

        ams.InitialBitrate = ams.AvailableBitrates.Max<uint>();

        //Register for download requests
        ams.DownloadRequested += DownloadRequested;

        //Register for download failure and completion events
        ams.DownloadCompleted += DownloadCompleted;
```



```

ams.DownloadFailed += DownloadFailed;

//Register for bitrate change events
ams.DownloadBitrateChanged += DownloadBitrateChanged;
ams.PlaybackBitrateChanged += PlaybackBitrateChanged;

//Register for diagnostic event
ams.Diagnostics.DiagnosticAvailable += DiagnosticAvailable;
}
else
{
    // Handle failure to create the adaptive media source
    MyLogMessageFunction($"Adaptive source creation failed: {uri} -
{result.ExtendedError}");
}
}

```

Initialize an AdaptiveMediaSource using HttpClient

If you need to set custom HTTP headers for getting the manifest file, you can create an [HttpClient](#) object, set the desired headers, and then pass the object into the overload of [CreateFromUriAsync](#).

C#

```

httpClient = new Windows.Web.Http.HttpClient();
httpClient.DefaultRequestHeaders.TryAppendWithoutValidation("X-
CustomHeader", "This is a custom header");
AdaptiveMediaSourceCreationResult result = await
AdaptiveMediaSource.CreateFromUriAsync(manifestUri, httpClient);

```

The [DownloadRequested](#) event is raised when the system is about to retrieve a resource from the server. The [AdaptiveMediaSourceDownloadRequestedEventArgs](#) passed into the event handler exposes properties that provide information about the resource being requested such as the type and URI of the resource.

Modify resource request properties using the DownloadRequested event

You can use the [DownloadRequested](#) event handler to modify the resource request by updating the properties of the [AdaptiveMediaSourceDownloadResult](#) object provided by the event args. In the example below, the URI from which the resource will be retrieved is modified by updating the [ResourceUri](#) properties of the result object. You

can also rewrite the byte range offset and length for media segments or, as shown the example below, change the resource URI to download the full resource and set the byte range offset and length to null.

You can override the content of the requested resource by setting the **Buffer** or **InputStream** properties of the result object. In the example below, the contents of the manifest resource are replaced by setting the **Buffer** property. Note that if you are updating the resource request with data that is obtained asynchronously, such as retrieving data from a remote server or asynchronous user authentication, you must call **AdaptiveMediaSourceDownloadRequestedEventArgs.GetDeferral** to get a deferral and then call **Complete** when the operation is complete to signal the system that the download request operation can continue.

C#

```
private async void DownloadRequested(AdaptiveMediaSource sender,
AdaptiveMediaSourceDownloadRequestedEventArgs args)
{
    // rewrite key URIs to replace http:// with https://
    if (args.ResourceType == AdaptiveMediaSourceResourceType.Key)
    {
        string originalUri = args.ResourceUri.ToString();
        string secureUri = originalUri.Replace("http:", "https:");

        // override the URI by setting property on the result sub object
        args.Result.ResourceUri = new Uri(secureUri);
    }

    if (args.ResourceType == AdaptiveMediaSourceResourceType.Manifest)
    {
        AdaptiveMediaSourceDownloadRequestedDeferral deferral =
args.GetDeferral();
        args.Result.Buffer = await
CreateMyCustomManifest(args.ResourceUri);
        deferral.Complete();
    }

    if (args.ResourceType ==
AdaptiveMediaSourceResourceType.MediaSegment)
    {
        var resourceUri = args.ResourceUri.ToString() + "?range=" +
args.ResourceByteRangeOffset + "-" +
(args.ResourceByteRangeLength - 1);

        // override the URI by setting a property on the result sub
object
        args.Result.ResourceUri = new Uri(resourceUri);

        // clear the byte range properties on the result sub object
        args.Result.ResourceByteRangeOffset = null;
    }
}
```

```
        args.Result.ResourceByteRangeLength = null;
    }
}
```

Use bitrate events to manage and respond to bitrate changes

The **AdaptiveMediaSource** object provides events that allow you to react when the download or playback bitrates change. In this example, the current bitrates are simply updated in the UI. Note that you can modify the ratios that determine when the system switches bitrates of the adaptive stream. For more information, see the [AdvancedSettings](#) property.

C#

```
private async void DownloadBitrateChanged(AdaptiveMediaSource sender,
AdaptiveMediaSourceDownloadBitrateChangedEventArgs args)
{
    await this.Dispatcher.RunAsync(CoreDispatcherPriority.Normal, new
DispatchedHandler(() =>
    {
        txtDownloadBitrate.Text = args.NewValue.ToString();
    }));
}

private async void PlaybackBitrateChanged(AdaptiveMediaSource sender,
AdaptiveMediaSourcePlaybackBitrateChangedEventArgs args)
{
    await this.Dispatcher.RunAsync(CoreDispatcherPriority.Normal, new
DispatchedHandler(() =>
    {
        txtPlaybackBitrate.Text = args.NewValue.ToString();
    }));
}
```

Handle download completion and failure events

The **AdaptiveMediaSource** object raises the **DownloadFailed** event when the download of a requested resource fails. You can use this event to update your UI in response to the failure. You can also use the event to log statistical information about the download operation and the failure.

The [AdaptiveMediaSourceDownloadFailedEventArgs](#) object passed into the event handler contains metadata about the failed resource download, such as the resource type, the URI of the resource, and the position within the stream where the failure occurred. The [RequestId](#) gets a system-generated unique identifier for the request that can be used to correlate status information about an individual request across multiple events.

The [Statistics](#) property returns a [AdaptiveMediaSourceDownloadStatistics](#) object which provides detailed information about the number of bytes received at the time of the event and the timing of various milestones in the download operation. You can log this information in order to identify performance issues in your adaptive streaming implementation.

C#

```
private void DownloadFailed(AdaptiveMediaSource sender,
    AdaptiveMediaSourceDownloadFailedEventArgs args)
{
    var statistics = args.Statistics;

    MyLogMessageFunction("download failed for: " + args.ResourceType +
        " - " + args.ResourceUri +
        " - Error:" + args.ExtendedError.HResult +
        " - RequestId" + args.RequestId +
        " - Position:" + args.Position +
        " - Duration:" + args.ResourceDuration +
        " - ContentType:" + args.ResourceContentType +
        " - TimeToHeadersReceived:" + statistics.TimeToHeadersReceived +
        " - TimeToFirstByteReceived:" + statistics.TimeToFirstByteReceived +
        " - TimeToLastByteReceived:" + statistics.TimeToLastByteReceived +
        " - ContentBytesReceivedCount:" +
        statistics.ContentBytesReceivedCount);
}
```

The [DownloadCompleted](#) event occurs when a resource download completes and provides similar data to the [DownloadFailed](#) event. Once again, a [RequestId](#) is provided for correlating events for a single request. Also, an [AdaptiveMediaSourceDownloadStatistics](#) object is provided to enable logging of download stats.

C#

```
private void DownloadCompleted(AdaptiveMediaSource sender,
    AdaptiveMediaSourceDownloadCompletedEventArgs args)
{
    var statistics = args.Statistics;
```

```

        MyLogMessageFunction("download completed for: " + args.ResourceType + "
- " +
        args.ResourceUri +
        " - RequestId:" + args.RequestId +
        " - Position:" + args.Position +
        " - Duration:" + args.ResourceDuration +
        " - ContentType:" + args.ResourceContentType +
        " - TimeToHeadersReceived:" + statistics.TimeToHeadersReceived +
        " - TimeToFirstByteReceived:" + statistics.TimeToFirstByteReceived +
        " - TimeToLastByteReceived:" + statistics.TimeToLastByteReceived +
        " - ContentBytesReceivedCount:" +
statistics.ContentBytesReceivedCount);
    }

```

Gather adaptive streaming telemetry data with AdaptiveMediaSourceDiagnostics

The **AdaptiveMediaSource** exposes a **Diagnostics** property which returns an **AdaptiveMediaSourceDiagnostics** object. Use this object to register for the **DiagnosticAvailable** event. This event is intended to be used for telemetry collection and should not be used to modify app behavior at runtime. This diagnostic event is raised for many different reasons. Check the **DiagnosticType** property of the **AdaptiveMediaSourceDiagnosticAvailableEventArgs** object passed into the event to determine the reason that the event was raised. Potential reasons include errors accessing the requested resource and errors parsing the streaming manifest file. For a list of situations that can trigger a diagnostic event, see **AdaptiveMediaSourceDiagnosticType**. Like the arguments for other adaptive streaming events, the **AdaptiveMediaSourceDiagnosticAvailableEventArgs** provides a **RequestId** property for correlating request information between different events.

C#

```

private void DiagnosticAvailable(AdaptiveMediaSourceDiagnostics sender,
AdaptiveMediaSourceDiagnosticAvailableEventArgs args)
{
    MySendTelemetryFunction(args.RequestId, args.Position,
                           args.DiagnosticType, args.SegmentId,
                           args.ResourceType, args.ResourceUri,
                           args.ResourceDuration, args.ResourceContentType,
                           args.ResourceByteRangeOffset,
                           args.ResourceByteRangeLength,
                           args.Bitrate,
                           args.ExtendedError);
}

```

Defer binding of adaptive streaming content for items in a playback list by using MediaBinder

The [MediaBinder](#) class allows you to defer binding of media content in a [MediaPlaybackList](#). Starting with Windows 10, version 1703, you can supply an [AdaptiveMediaSource](#) as bound content. The process for deferred binding of an adaptive media source is largely the same as binding other types of media, which is described in [Media items, playlists, and tracks](#).

Create a **MediaBinder** instance, set an app-defined **Token** string to identify the content to be bound, and register for the **Binding** event. Create a **MediaSource** from the **Binder** by calling [MediaSource.CreateFromMediaBinder](#). Then, create a **MediaPlaybackItem** from the **MediaSource** and add it to the playback list.

C#

```
_mediaPlaybackList = new MediaPlaybackList();

var binder = new MediaBinder();
binder.Token = "MyBindingToken1";
binder.Binding += Binder_Binding;
_mediaPlaybackList.Items.Add(new
MediaPlaybackItem(MediaSource.CreateFromMediaBinder(binder)));

binder = new MediaBinder();
binder.Token = "MyBindingToken2";
binder.Binding += Binder_Binding;
_mediaPlaybackList.Items.Add(new
MediaPlaybackItem(MediaSource.CreateFromMediaBinder(binder)));

_mediaPlayer = new MediaPlayer();
_mediaPlayer.Source = _mediaPlaybackList;
mediaPlayerElement.SetMediaPlayer(_mediaPlayer);
```

In the **Binding** event handler, use the token string to identify the content to be bound and then create the adaptive media source by calling one of the overloads of [CreateFromStreamAsync](#) or [CreateFromUriAsync](#). Because these are asynchronous methods, you should first call the [MediaBindingEventArgs.GetDeferral](#) method to instruct the system to wait for your operation to complete before continuing. Set the adaptive media source as the bound content by calling [SetAdaptiveMediaSource](#). Finally, call [Deferral.Complete](#) after your operation is complete to instruct the system to continue.

C#

```

private async void Binder_Binding_AdaptiveMediaSource(MediaBinder sender,
MediaBindingEventArgs args)
{
    var deferral = args.GetDeferral();

    var contentUri = new
Uri($"http://contoso.com/media/{args.MediaBinder.Token}");
    AdaptiveMediaSourceCreationResult result = await
AdaptiveMediaSource.CreateFromUriAsync(contentUri);

    if (result.MediaSource != null)
    {
        args.SetAdaptiveMediaSource(result.MediaSource);
    }
    args.SetUri(contentUri);

    deferral.Complete();
}

```

If you want to register event handlers for the bound adaptive media source, you can do this in the handler for the [CurrentItemChanged](#) event of the **MediaPlaybackList**. The [CurrentMediaPlaybackItemChangedEventArgs.NewItem](#) property contains the new currently playing **MediaPlaybackItem** in the list. Get an instance of the **AdaptiveMediaSource** representing the new item by accessing the [Source](#) property of the **MediaPlaybackItem** and then the [AdaptiveMediaSource](#) property of the media source. This property will be null if the new playback item is not an **AdaptiveMediaSource**, so you should test for null before attempting to register handlers for any of the object's events.

C#

```

private void AMSMediaPlaybackList_CurrentItemChanged(MediaPlaybackList
sender, CurrentMediaPlaybackItemChangedEventArgs args)
{
    if (!(args.NewItem is null))
    {
        var ams = args.NewItem.Source.AdaptiveMediaSource;
        if (!(ams is null))
        {
            ams.PlaybackBitrateChanged += Ams_PlaybackBitrateChanged;
        }
    }
}

```

Related topics

- [Media playback](#)

- [HLS tag support](#)
 - [Dash profile support](#)
 - [Play audio and video with MediaPlayer](#)
 - [Play media in the background](#)
-

Feedback

Was this page helpful?

 Yes

 No

[Provide product feedback](#)  | [Get help at Microsoft Q&A](#)

HTTP Live Streaming (HLS) tag support

Article • 10/20/2022

The following table lists the HLS tags that are supported for UWP apps.

ⓘ Note

Custom tags that start with "X-" can be accessed as timed metadata as described in the article [Media items, playlists, and tracks](#).

Tag	Introduced in HLS Protocol version	HLS Protocol Document Draft Version	Required on Client	July release of Windows 10	Windows 10, Version 1511	Windows 10, Version 1607
4.3.1. Basic Tags						
4.3.1.1. EXT-M3U	1	0	REQUIRED	Supported	Supported	Supported
4.3.1.2. EXT-X-VERSION	2	3	REQUIRED	Supported	Supported	Supported
4.3.2. Media Segment Tags						
4.3.2.1. EXTINF	1	0	REQUIRED	Supported	Supported	Supported
4.3.2.2. EXT-X-BYTERANGE	4	7	OPTIONAL	Supported	Supported	Supported
4.3.2.3. EXT-X-DISCONTINUITY	1	2	OPTIONAL	Supported	Supported	Supported
4.3.2.4. EXT-X-KEY	1	0	OPTIONAL	Supported	Supported	Supported
METHOD	1	0	Attribute	"NONE, AES-128"	"NONE, AES-128"	"NONE, AES-128, SAMPLE-AES"
URI	1	0	Attribute	Supported	Supported	Supported
IV	2	3	Attribute	Supported	Supported	Supported
KEYFORMAT	5	9	Attribute	Not Supported	Not Supported	Not Supported

Tag	Introduced in HLS Protocol version	HLS Protocol Document Draft Version	Required on Client	July release of Windows 10	Windows 10, Version 1511	Windows 10, Version 1607
KEYFORMATVERSIONS	5	9	Attribute	Not Supported	Not Supported	Not Supported
4.3.2.5. EXT-X-MAP	5	9	OPTIONAL	Not Supported	Not Supported	Not Supported
URI	5	9	Attribute	Not Supported	Not Supported	Not Supported
BYTERANGE	5	9	Attribute	Not Supported	Not Supported	Not Supported
4.3.2.6. EXT-X-PROGRAM-DATE-TIME	1	0	OPTIONAL	Not Supported	Not Supported	Not Supported
4.3.3. Media Playlist Tags						
4.3.3.1. EXT-X-TARGETDURATION	1	0	REQUIRED	Supported	Supported	Supported
4.3.3.2. EXT-X-MEDIA-SEQUENCE	1	0	OPTIONAL	Supported	Supported	Supported
4.3.3.3. EXT-X-DISCONTINUITY-SEQUENCE	6	12	OPTIONAL	Not Supported	Not Supported	Not Supported
4.3.3.4. EXT-X-ENDLIST	1	0	OPTIONAL	Supported	Supported	Supported
4.3.3.5. EXT-X-PLAYLIST-TYPE	3	6	OPTIONAL	Supported	Supported	Supported
4.3.3.6. EXT-X-I-FRAMES-ONLY	4	7	OPTIONAL	Not Supported	Not Supported	Not Supported
4.3.4. Master Playlist Tags						
4.3.4.1. EXT-X-MEDIA	4	7	OPTIONAL	Supported	Supported	Supported
TYPE	4	7	Attribute	"AUDIO, VIDEO"	"AUDIO, VIDEO"	"AUDIO, VIDEO, SUBTITLES"
URI	4	7	Attribute	Supported	Supported	Supported

Tag	Introduced in HLS Protocol version	HLS Protocol Document Draft Version	Required on Client	July release of Windows 10	Windows 10, Version 1511	Windows 10, Version 1607
GROUP-ID	4	7	Attribute	Supported	Supported	Supported
LANGUAGE	4	7	Attribute	Supported	Supported	Supported
ASSOC-LANGUAGE	6	13	Attribute	Not Supported	Not Supported	Not Supported
NAME	4	7	Attribute	Not Supported	Not Supported	Supported
DEFAULT	4	7	Attribute	Not Supported	Not Supported	Not Supported
AUTOSELECT	4	7	Attribute	Not Supported	Not Supported	Not Supported
FORCED	5	9	Attribute	Not Supported	Not Supported	Not Supported
INSTREAM-ID	6	12	Attribute	Not Supported	Not Supported	Not Supported
CHARACTERISTICS	5	9	Attribute	Not Supported	Not Supported	Not Supported
4.3.4.2. EXT-X- STREAM-INF	1	0	REQUIRED	Supported	Supported	Supported
BANDWIDTH	1	0	Attribute	Supported	Supported	Supported
PROGRAM-ID	1	0	Attribute	NA	NA	NA
AVERAGE- BANDWIDTH	7	14	Attribute	Not Supported	Not Supported	Not Supported
CODECS	1	0	Attribute	Supported	Supported	Supported
RESOLUTION	2	3	Attribute	Supported	Supported	Supported
FRAME-RATE	7	15	Attribute	NA	NA	NA
AUDIO	4	7	Attribute	Supported	Supported	Supported
VIDEO	4	7	Attribute	Supported	Supported	Supported

Tag	Introduced in HLS Protocol version	HLS Protocol Document Draft Version	Required on Client	July release of Windows 10	Windows 10, Version 1511	Windows 10, Version 1607
SUBTITLES	5	9	Attribute	Not Supported	Not Supported	Supported
CLOSED-CAPTIONS	6	12	Attribute	Not Supported	Not Supported	Not Supported
4.3.4.3. EXT-X-I- FRAME-STREAM-INF	4	7	OPTIONAL	Not Supported	Not Supported	Not Supported
4.3.4.4. EXT-X- SESSION-DATA	7	14	OPTIONAL	Not Supported	Not Supported	Not Supported
4.3.4.5. EXT-X- SESSION-KEY	7	17	OPTIONAL	Not Supported	Not Supported	Not Supported
4.3.5. Media or Master Playlist Tags						
4.3.5.1. EXT-X- INDEPENDENT- SEGMENTS	6	13	OPTIONAL	Not Supported	Supported	Supported
4.3.5.2. EXT-X-START	6	12	OPTIONAL	Not Supported	Partially Supported	Partially Supported
TIME-OFFSET	6	12	Attribute	Not Supported	Supported	Supported
PRECISE	6	12	Attribute	Not Supported	Default "NO" supported	Default "NO" supported

Related topics

- [Media playback](#)
- [Adaptive streaming](#)

Dynamic Adaptive Streaming over HTTP (DASH) profile support

Article • 10/20/2022

Supported DASH profiles

The following table lists the DASH profiles that are supported for UWP apps.

Tag	Manifest type	Notes	July release of Windows 10	Windows 10, Version 1511	Windows 10, Version 1607	Windows 10, Version 1607	Windows 10, Version 1703	Windows 10, Version 1809
urn:mpeg:dash:profile:isoff-live:2011	Static		Supported	Supported	Supported	Supported	Supported	Supported
urn:mpeg:dash:profile:isoff-main:2011		Best effort	Supported	Supported	Supported	Supported	Supported	Supported
urn:mpeg:dash:profile:isoff-live:2011	Dynamic	\$Time\$ is supported but \$Number\$ is unsupported in segment templates	Not Supported	Not Supported	Not Supported	Not Supported	Supported	Supported
urn:mpeg:dash:profile:isoff-on-demand:2011			Not Supported	Not Supported	Not Supported	Not Supported	Not Supported	Supported

Unsupported DASH profiles

Profiles not listed in the above table are unsupported, including but not limited to the following:

- urn:mpeg:dash:profile:full:2011
- urn:mpeg:dash:profile:mp2t-main:2011
- urn:mpeg:dash:profile:mp2t-simple:2011

Related topics

- [Media playback](#)
- [Adaptive streaming](#)